

Source Code – Lost And Found

(Web Based Item Recovery System)

Computer Software – Copyright Submission

Introduction

This document presents selected core source code of the **Lost & Found Portal**, a web-based item management system designed to help users report, search, and recover lost and found items efficiently. The system provides a centralized platform with features such as item posting, image upload, category-based filtering, messaging, and admin approval to ensure reliability and transparency.

The included source code highlights the primary architecture, key functionalities, and implementation logic of the application. Only essential modules are provided to demonstrate originality, system design, and core operational features of the system.

BACKEND

File: server/app.js

```
import express from 'express'
import mongoose from 'mongoose'
import morgan from 'morgan'
import bodyParser from 'body-parser'
import cookieParser from 'cookie-parser'
import cors from 'cors'
import dotenv from 'dotenv'

import userRoutes from './routes/userRoutes.js'
import ItemRoutes from './routes/ItemRoutes.js'

const app = express()
dotenv.config();
app.use(cors({ origin: 'http://localhost:3000', credentials: true })))
app.use(express.json({ limit: '20mb' })))
app.use(morgan('dev'))
app.use(bodyParser.urlencoded({ extended: true, limit: '20mb' })))
app.use(bodyParser.json({ limit: '20mb' })))
app.use(cookieParser())

app.use((_req, res, next) => {
  res.header('Access-Control-Allow-Origin', 'http://localhost:3000')
  res.header('Access-Control-Allow-Credentials', 'true')
  res.header(
    'Access-Control-Allow-Headers',
    'Origin, X-Requested-With, Content-Type, Accept, token, Authorization'
  )
  res.header(
    'Access-Control-Allow-Methods',
    'GET, POST, OPTIONS, PUT, DELETE'
  )
  next()
})

app.use('/users', userRoutes)
app.use('/Items', ItemRoutes)
app.use('/items', ItemRoutes)
```

```
const port = process.env.PORT || 5000;
const db = process.env.MONGO_URI;

mongoose.connect(db, { useNewUrlParser: true, useUnifiedTopology: true }).then(()
=> app.listen(port, () => console.log('Connection done and running on PORT :'+
port))).catch((err) => console.log(err.message));
```

File: server/user/CreateUser.js

```
import User from '../models/User.js'
import bcrypt from 'bcryptjs'
import jwt from 'jsonwebtoken'
import dotenv from 'dotenv'
dotenv.config();

const secretKey = process.env.JWT_SECRET

const generateJWT = async (id) => {
  const token = jwt.sign({ id }, secretKey, {
    expiresIn: '24h',
  })
  return token
}

const createUser = async (req, res) => {
  const userData = req.body

  const data = {
    members: [
      {
        email_address: userData.email,
      },
    ],
  },
```

```

    }

    const postData = JSON.stringify(data)

    try {
      const findUser = await User.findOne({ email: userData.email })
      if (findUser) {
        return res.status(200).json({
          ok: false,
          msg: 'The email is already used',
        })
      }

      const newUser = new User(userData)

      //Encrypt password
      const salt = bcrypt.genSaltSync()
      newUser.password = bcrypt.hashSync(newUser.password, salt)

      await newUser.save()
      const token = await generateJWT(newUser.id)
      return res.status(201).json({
        ok: true,
        msg: 'User Created',
        user: newUser,
        token,
      })
    } catch (error) {
      console.log(error)
      return res.status(500).json({
        ok: false,
        msg: 'An error occurred, contact an administrator',
      })
    }
  }
}

export default createUser

```

File: server/User/LoginUser.js

```
import User from '../models/User.js'
import bcrypt from 'bcryptjs'
import jwt from 'jsonwebtoken'
import dotenv from 'dotenv'
dotenv.config();

const secretKey = process.env.JWT_SECRET

const generateJWT = async (id) => {
  const token = jwt.sign({ id }, secretKey, {
    expiresIn: '24h',
  })
  return token
}

const createUser = async (req,res) => {
  const userData = req.body

  const data = {
    members: [
      {
        email_address: userData.email,
      },
    ],
  }

  const postData = JSON.stringify(data)

  try {
    const findUser = await User.findOne({ email: userData.email })
    if (findUser) {
      return res.status(200).json({
        ok: false,
        msg: 'The email is already used',
      })
    }
  }

  const newUser = new User(userData)

  //Encrypt password
  const salt = bcrypt.genSaltSync()
  newUser.password = bcrypt.hashSync(newUser.password, salt)

  await newUser.save()
  const token = await generateJWT(newUser.id)
```

```

    return res.status(201).json({
      ok: true,
      msg: 'User Created',
      user: newUser,
      token,
    })
  } catch (error) {
    console.log(error)
    return res.status(500).json({
      ok: false,
      msg: 'An error occurred, contact an administrator',
    })
  }
}

export default createUser

```

File:server/User/renewToken.js

```

import jwt from 'jsonwebtoken'
import dotenv from 'dotenv'
dotenv.config();

const secretKey = process.env.JWT_SECRET

const generateJWT = async (id) => {
  const token = jwt.sign({ id }, secretKey, {
    expiresIn: '24h',
  })
  return token
}

export const renewToken = async (req, res) => {
  const id = req.id

  const token = await generateJWT(id)

  res.status(200).json({ ok: true, id, token: token })
}

```

File: server/User/UpdateUser.js

```
import User from '../models/User.js'

export const updateUser = async (req, res) => {
  const { id } = req.params
  const { nickname, fullname, img, number } = req.body
  const newData = { nickname, fullname, img, number }

  try {
    if (req.id !== id) {
      return res.status(403).json({
        ok: false,
        msg: 'You can only update your own profile',
      })
    }

    const userUpdated = await User.findOneAndUpdate({ _id: id }, newData, {
      new: true,
    })

    if (userUpdated) {
      return res
        .status(200)
        .json({ ok: true, msg: 'User Updated!', userUpdated })
    }

    return res.status(404).json({
      ok: false,
      msg: 'The user that you're trying to edit doesn't exist',
    })
  } catch (error) {
    console.log(error)
    return res.status(500).json({
      ok: false,
      msg: 'An error occurred, contact an administrator',
    })
  }
}
```

MODELS

File: server/models/Item.js

```
import jwt from 'jsonwebtoken'
import dotenv from 'dotenv'
dotenv.config();

const secretKey = process.env.JWT_SECRET

export const validateJWT = async (req, res, next) => {

  const token = req.header('token')

  try {
    if (!token) {
      return res.status(401).json({
        ok: false,
        msg: 'Access denied',
      })
    }

    const payload = jwt.verify(token, secretKey)

    req.id = payload.id
    return next()
  } catch (error) {
    console.log(error)
    return res.status(401).json({
      ok: false,
      msg: 'Token not valid',
    })
  }
}

export default validateJWT;
```

File: server/models/User.js

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  nickname: {
    type: String,
    required: true,
  },
  fullname: {
    type: String,
  },
  img: {
    type: String,
    default:
      'https://www.softzone.es/app/uploads-softzone.es/2018/04/guest.png',
  },
  password: {
    type: String,
  },
  email: {
    type: String,
    unique: true,
    required: true,
  },
})

const User = mongoose.model('User', userSchema)
export default User
```

MIDDLEWARE

File: server/middlewares/validateToken.js

```
import jwt from 'jsonwebtoken'
import dotenv from 'dotenv'
dotenv.config();

const secretKey = process.env.JWT_SECRET

export const validateJWT = async (req, res, next) => {

  const token = req.header('token')

  try {
    if (!token) {
      return res.status(401).json({
        ok: false,
        msg: 'Access denied',
      })
    }

    const payload = jwt.verify(token, secretKey)

    req.id = payload.id
    return next()
  } catch (error) {
    console.log(error)
    return res.status(401).json({
      ok: false,
      msg: 'Token not valid',
    })
  }
}

export default validateJWT;
```

FRONTEND

File: frontend/src/app.js

```
import React from "react";
import { BrowserRouter, Route, Routes } from "react-router-dom";
import { ToastContainer } from 'react-toastify';
import { ThemeProvider } from '@mui/material/styles';
import { CssBaseline } from "@mui/material";
import Signup from "./Components/Signup";
import Login from "./Components/Login";
import LostItems from "./Components/LostItems";
import FoundItems from "./Components/FoundItems";
import StolenItems from "./Components/StolenItems";
import Home from "./Components/Home";
import Dashboard from "./Components/Dashboard";
import ItemPage from "./Components/ItemPage";
import LostItem from "./Components/Lost_item";
import MyListings from "./Components/MyListings";
import HowItWorks from "./Components/HowItWorks";
import Profile from "./Components/Profile";
import ProtectedRoute from "./Components/ProtectedRoute";
import AuthRoute from "./Components/AuthRoute";
import Layout from "./layout";
import finditTheme from './theme';

function App() {
  const token = window.localStorage.getItem("token");

  return (
    <ThemeProvider theme={finditTheme}>
      <CssBaseline />
      <BrowserRouter>
        <Layout>
          <Routes>

            <Route path="/" element={token ? <Dashboard /> : <Home />} />
            <Route path="/dashboard" element={<AuthRoute><Dashboard
/></AuthRoute>} />
            <Route path="/log-in" element={<ProtectedRoute><Login
/></ProtectedRoute>} />
            <Route path="/sign-up" element={<ProtectedRoute><Signup
```

```

/></ProtectedRoute>} />
    <Route path="/lostitems"
element={<AuthRoute><LostItems/></AuthRoute>} />
    <Route path="/founditems"
element={<AuthRoute><FoundItems/></AuthRoute>} />
    <Route path="/stolenitems"
element={<AuthRoute><StolenItems/></AuthRoute>} />
    <Route path="/postitem" element={<AuthRoute><LostItem
/></AuthRoute>} />
    <Route path="/mylistings" element={<AuthRoute><MyListings
/></AuthRoute>} />
    <Route path="/how-it-works" element={<AuthRoute><HowItWorks
/></AuthRoute>} />
    <Route path="/profile" element={<AuthRoute><Profile /></AuthRoute>}
/>
    <Route path="/item/:id"
element={<AuthRoute><ItemPage/></AuthRoute>} />
    <Route path="/*" element={token ? <Dashboard /> : <Home/>} />
  </Routes>
  <ToastContainer />
  </Layout>
</BrowserRouter>
</ThemeProvider>

);
}

export default App;

```

File: server/src/api.js

```

import axios from "axios";

const API_BASE_URL = process.env.REACT_APP_API_URL ||
"http://localhost:4000";

export const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
});

```

```
export const authHeaders = () => {  
  const token = localStorage.getItem("token");  
  return token ? { token } : {};  
};
```

```
export default api;
```

File: client/src/index.js

```
-----
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import CssBaseline from '@mui/material/CssBaseline';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(

  <<CssBaseline /><App /></>

);
```

File: client/src/layout.js

```
-----
import React from "react";
import { Stack, Box, Container } from '@mui/material'
import { useLocation } from "react-router-dom";
import Navbar from "./Components/Navbar.js";
import Footer from "./Components/footer.js";

function Layout(props) {
  const token = window.localStorage.getItem("token");
  const location = useLocation();
  const publicFooterRoutes = ["/", "/log-in", "/sign-up"];
  const showFooter = !token && publicFooterRoutes.includes(location.pathname);

  return (
    <Stack
      spacing="0"
      width="100%"
      alignItems="stretch"
      minHeight="100vh"
      justifyContent="flex-start"
    >
      <Navbar />
      <Box sx={{ flex: 1, width: '100%', overflow: 'hidden', py: { xs: 2, md: 3 } }}>
        <Container maxWidth="xl" sx={{ px: { xs: 2, md: 4 } }}>
          {props.children}
        </Container>
      </Box>
      {showFooter ? <Footer /> : null}
    </Stack>
  );
}
```

```
export default Layout;
```

File:client/src/theme.js

```
import { alpha, createTheme } from "@mui/material/styles";

export const finditTheme = createTheme({
  palette: {
    mode: "light",
    primary: {
      main: "#0ea5e9",
      light: "#38bdf8",
      dark: "#0284c7",
      contrastText: "#ffffff",
    },
    secondary: {
      main: "#0f766e",
      light: "#14b8a6",
      dark: "#115e59",
      contrastText: "#ffffff",
    },
    background: {
      default: "#f0f9ff",
      paper: "#ffffff",
    },
    text: {
      primary: "#0f172a",
      secondary: "#475569",
    },
    success: { main: "#16a34a" },
    warning: { main: "#d97706" },
    error: { main: "#dc2626" },
    info: { main: "#0ea5e9" },
    divider: alpha("#0f172a", 0.1),
  },
  shape: {
    borderRadius: 14,
  },
  typography: {
    fontFamily: '"Inter", "Poppins", "Roboto", "Helvetica", "Arial", sans-serif',
    h1: { fontWeight: 800, letterSpacing: "-0.02em" },
    h2: { fontWeight: 700, letterSpacing: "-0.01em" },
    h3: { fontWeight: 700 },
    h4: { fontWeight: 650 },
    h5: { fontWeight: 650 },
    h6: { fontWeight: 600 },
  },
});
```

```

    button: { fontWeight: 600 },
  },
  components: {
    MuiCssBaseline: {
      styleOverrides: {
        body: {
          background:
            "radial-gradient(circle at 10% -10%, rgba(14, 165, 233, 0.12) 0%,
transparent 32%), radial-gradient(circle at 90% 10%, rgba(56, 189, 248, 0.12) 0%,
transparent 30%), #f0f9ff",
        },
      },
    },
    MuiContainer: {
      defaultProps: {
        maxWidth: "lg",
      },
    },
    MuiAppBar: {
      styleOverrides: {
        root: {
          backgroundColor: alpha("#ffffff", 0.9),
          backdropFilter: "blur(10px)",
          boxShadow: "0 10px 30px rgba(15, 23, 42, 0.06)",
          borderBottom: `1px solid ${alpha("#0f172a", 0.08)}`,
        },
      },
    },
    MuiPaper: {
      styleOverrides: {
        root: {
          borderRadius: 16,
          border: `1px solid ${alpha("#0f172a", 0.06)}`,
          boxShadow: "0 14px 36px rgba(15, 23, 42, 0.06)",
        },
      },
    },
    MuiCard: {
      styleOverrides: {
        root: {
          borderRadius: 18,
          border: `1px solid ${alpha("#0f172a", 0.06)}`,
          boxShadow: "0 12px 30px rgba(15, 23, 42, 0.06)",
          transition: "all 0.25s ease",
          "&:hover": {
            transform: "translateY(-4px)",
            boxShadow: "0 18px 34px rgba(15, 23, 42, 0.12)",
          },
        },
      },
    },
  },

```

```

    },
  },
},
MuiButton: {
  styleOverrides: {
    root: {
      textTransform: "none",
      borderRadius: 12,
      padding: "10px 18px",
      boxShadow: "none",
    },
    containedPrimary: {
      background: "linear-gradient(135deg, #0ea5e9 0%, #0284c7 100%)",
      boxShadow: "0 10px 24px rgba(14, 165, 233, 0.28)",
      "&:hover": {
        boxShadow: "0 14px 28px rgba(14, 165, 233, 0.38)",
      },
    },
  },
},
MuiTextField: {
  defaultProps: {
    size: "small",
  },
  styleOverrides: {
    root: {
      "& .MuiOutlinedInput-root": {
        backgroundColor: "#ffffff",
        borderRadius: 12,
        "& .Mui-focused .MuiOutlinedInput-notchedOutline": {
          borderWidth: 2,
        },
      },
    },
  },
},
MuiChip: {
  styleOverrides: {
    root: {
      borderRadius: 10,
      fontWeight: 600,
    },
  },
},
});

```

```
export default finditTheme;
```

End of Source Code Document

This document contains selected core implementation of the lost and found app
application.
